CSL301 - Operating Systems

Lab 6 - Submission Instructions

Submission Format:

- Submit all tasks in a single ZIP file named: `ID_Name.zip`
- Create separate subfolders for each task inside the ZIP file:

```
ID_Name.zip
|
|-- task1/
|
|-- task2/
|
|-- task3/
|
|-- task4/
```

- The individual source files should be named exactly as specified in the respective task.
- Each task folder should contain all the required source files for that task.
- Each task folder must also contain a **screenshot of the program output** demonstrating that the program was successfully executed.
- The screenshot should clearly show the relevant input, output, and program behavior required by the task.
- Ensure that the submitted programs compile and run correctly before submission.

Example Submission Structure:

```
ID_Name.zip
|
|-- task1/
|   |-- server_task1.c
|   |-- client_task1.c
|   |-- output.png
|
|-- task2/
|   |-- server_task2.c
|   |-- client_task2.c
|   |-- output.png
|
|-- task3/
|   |-- server_task3.c
|   |-- client_task3.c
|   |-- output.png
|
|-- task4/
|   |-- signal_task4.c
|   |-- output.png
```

CSL301 - Operating Systems

Lab 6

Task 1: TCP Unit Converter

Objective: Implement a TCP client-server Unit Converter in C. The client sends a conversion request to the server, the server performs the conversion, and sends the result back to the client.

Requirements:

- The server should create an IPv4 TCP socket and listen on port `8080`.
- The client should accept the server IP address as a command-line argument and connect to the server.
- The client should send conversion requests and display the server's response.
- The server should parse the request, perform the conversion, and send the result back.
- Multiple conversion requests should be handled over the **same TCP connection**.
- The command `EXIT` should terminate the current client session, while the server continues waiting for new clients.
- Invalid or incorrectly formatted requests should be handled without crashing the program.

Supported Requests:

```
CM_TO_M <value>
M_TO_KM <value>
C_TO_F <value>
KG_TO_G <value>
EXIT
```

For example, a request such as `CM_TO_M 250` should result in a response equivalent to `250 cm = 2.50 m`.

Testing on the Same Machine:

```
gcc server_task1.c -o server_task1
gcc client_task1.c -o client_task1

# Terminal 1
./server_task1

# Terminal 2
./client_task1 127.0.0.1
```

Goal: Reuse the TCP socket programming concepts learned in the basic client-server program to build a new application.

Task 2: TCP-Based File Transfer Using Client-Server Sockets

Objective: Design and implement a TCP-based Client-Server File Transfer System in C that allows a client to upload files to the server and download files from the server over a reliable TCP connection.

Requirements:

- The server should create an IPv4 TCP socket and listen on port `8080`.
- The client should accept the server IP address as a command-line argument and connect to the server.
- The server should maintain a **persistent TCP connection** and allow multiple file-transfer operations without reconnecting.
- The client should provide the following menu-driven interface:

```
==================================
        FILE TRANSFER SYSTEM
==================================
1. Upload File
2. Download File
3. Exit
==================================
```

- For **Upload**, the client should read a file and send it to the server, where it should be saved.
- For **Download**, the client should request a file from the server, receive its contents, and save it locally.
- Files should be transferred using a **fixed-size buffer** and should support files larger than the buffer size.
- The server should handle invalid requests or missing files without crashing.
- The client should be able to perform multiple upload/download operations using the same TCP connection.
- The `Exit` option should terminate the client session and close the connection cleanly.

Testing on the Same Machine:

```
gcc server_task2.c -o server_task2
gcc client_task2.c -o client_task2

# Terminal 1
./server_task2

# Terminal 2
./client_task2 127.0.0.1
```

Goal: Apply TCP socket programming, file I/O, buffered data transfer, and persistent client-server communication to build a practical file-transfer application.

Task 3: Remote C Program Execution Using TCP Sockets

Objective: Modify the given TCP Client-Server C program so that the client sends a C source file to the server, the server compiles and executes the program, and the **output of the executed program is sent directly back to the client** instead of sending the generated executable file.

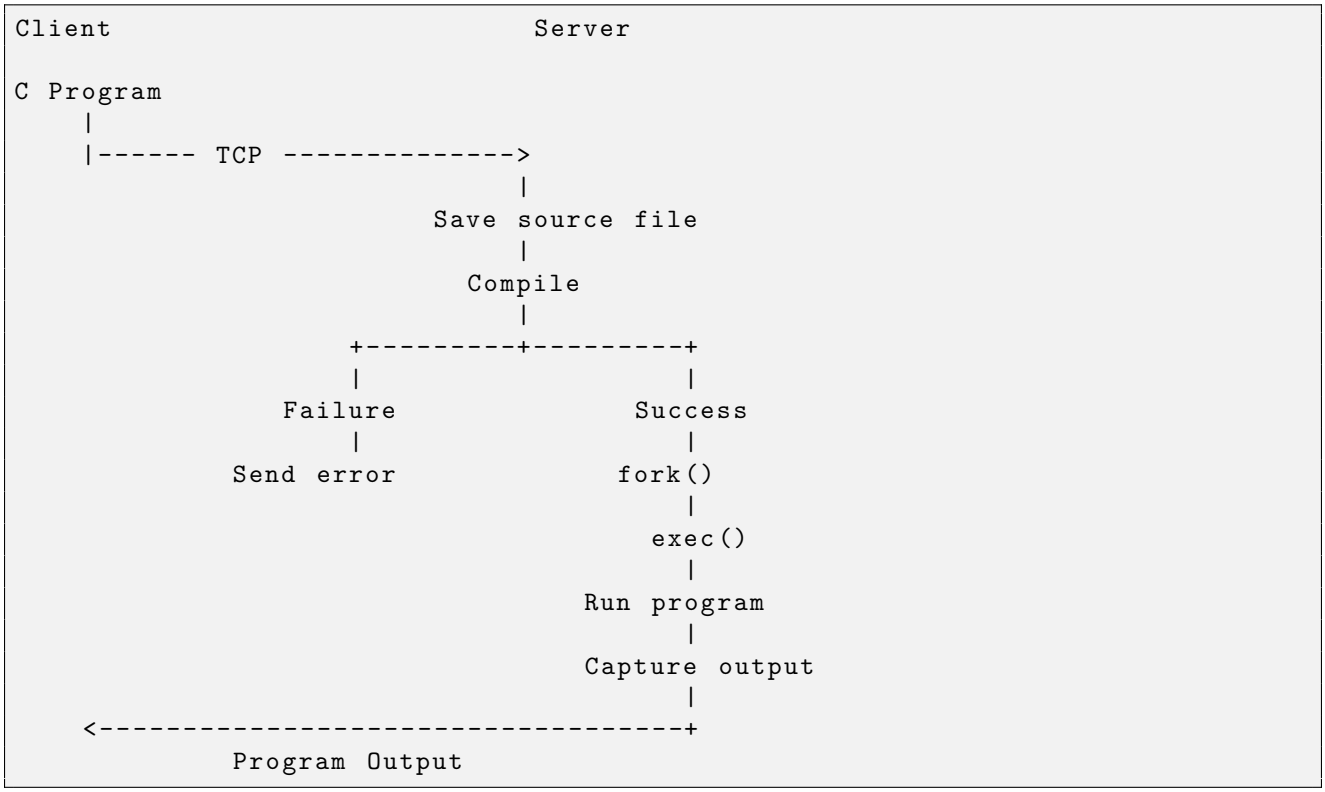
Requirements:

- The server should create an IPv4 TCP socket and listen on port `8080`.
- The client should accept the server IP address as a command-line argument and connect to the server.
- The client should send a C source file to the server.
- The server should receive and save the C source file before attempting to compile it.
- The server should compile the received C program using the `gcc` compiler.
- If compilation fails, the server should send the compilation errors back to the client.
- If compilation succeeds, the server should execute the generated program on the server.
- The server should **capture the output of the executed program** and send that output directly to the client.
- The generated executable should **not be sent to the client**.
- The server should use the `fork()`, `exec()` and `pipe()` concepts to create the child process, execute the compiled program, and capture its output.
- The client should display the output received from the server.
- If the submitted C program contains compilation errors, the client should display the compilation error message received from the server.
- The server should handle invalid requests and compilation failures without crashing.
- The server should clean up the received source file and generated executable after execution.
- The client should close the TCP connection cleanly after receiving the result.

Important Modification:

The basic program provided in class sends the generated executable file from the server back to the client after successful compilation. For this task, modify the program so that the server executes the compiled program and sends its **execution output directly to the client**.

Expected Flow:



Expected Output:

For example, if the client sends the following C program:

```
#include <stdio.h>

int main()
{
    printf("Hello from the server!\n");
    printf("Program executed successfully.\n");
    return 0;
}
```

The client should display something similar to:

```
Connected to server.
Sending file...
File sent.

Waiting for execution result...

Program Output:
--------------------------------
Hello from the server!
Program executed successfully.
--------------------------------
```

The executable generated on the server should not be downloaded or saved on the client.

Testing on the Same Machine:

```
gcc server_task3.c -o server_task3
gcc client_task3.c -o client_task3

# Terminal 1
./server_task3

# Terminal 2
./client_task3 127.0.0.1
```

Goal: Modify the given TCP file-transfer and remote compilation program to introduce process execution and inter-process communication. The server should use `fork()`, `exec()`, and `pipe()` to execute the uploaded C program, capture its output, and return the output directly to the client instead of transferring the executable file.

Task 4: Signal-Based Process Control

Objective: Design and implement a C program that demonstrates signal handling and process control using UNIX signals. The program should respond differently to different signals and allow the process to be paused, resumed, and terminated gracefully.

Requirements:

- The program should display its **process ID (PID)** when it starts.
- The program should continuously run a main loop and display its current state once every second.
- The program should register signal handlers using `signal()`.
- On receiving **SIGUSR1**, the program should **pause** its main activity.
- On receiving **SIGUSR2**, the program should **resume** its main activity.
- On receiving **SIGINT** (generated using `Ctrl+C`), the program should display a warning message but should **not terminate**.
- On receiving **SIGTERM**, the program should terminate the main loop and exit gracefully.
- The program should maintain a variable representing its current state, such as `RUNNING` or `PAUSED`.
- The state should be changed by the appropriate signal handlers.
- The program should continue running after receiving `SIGINT`, `SIGUSR1`, or `SIGUSR2`.
- The program should terminate only when `SIGTERM` is received.
- The program should handle the signals without crashing.

Expected Behavior:

```
Program started
PID: 12345

State: RUNNING
State: RUNNING
State: RUNNING

SIGUSR1 received
Process paused

State: PAUSED
State: PAUSED
State: PAUSED

SIGUSR2 received
Process resumed

State: RUNNING
State: RUNNING

Ctrl+C pressed
SIGINT received - process continues running

State: RUNNING
State: RUNNING

SIGTERM received
Terminating gracefully...
Program exited
```

Testing:

Compile and run the program:

```
gcc signal_task4.c -o signal_task4

./signal_task4
```

The program will display its PID. Open another terminal and use the following commands, replacing `12345` with the actual PID of the running program:

```
# Pause the process
kill -SIGUSR1 12345

# Resume the process
kill -SIGUSR2 12345

# Terminate the process gracefully
kill -SIGTERM 12345
```

Students should also press `Ctrl+C` in the program terminal to verify that `SIGINT` is caught without terminating the process.

Goal: Apply the signal-handling concepts learned in class, including `signal()`, signal handlers, `SIGINT`, `SIGUSR1`, `SIGUSR2`, `SIGTERM`, process IDs, and graceful process termination to implement a signal-controlled process.